JFEM Automatic Differentiation, Adjoint Sensitivity, Optimization, and SOL 200 Implementation

Technical Code Documentation

JFEM development workspace

Generated from source survey on 2026-06-01

Abstract

This document describes the sensitivity and optimization capabilities implemented in JFEM: targeted automatic differentiation, direct derivative kernels, static and buckling adjoint solvers, grouped sizing optimization, and the current SOL 200-lite translation route. The text is based on the implemented source code, especially `src/solver/dKdx.jl`, `src/solver/adjoint.jl`, `src/solver/adjoint_buckling.jl`, `src/solver/optimize_thickness.jl`, `src/JFEMSolver.jl`, and `src/main.jl`. The goal is to give future developers and users a clear mental model of what is already implemented, how the pieces interact, and where the supported boundary currently lies.

Contents

1 Executive Summary

JFEM contains a practical sensitivity and optimization stack rather than a single monolithic "differentiate everything" mechanism. The implemented design is layered:

- **Targeted automatic differentiation**: local element derivatives are computed with `ForwardDiff` where it is accurate and economical, mainly inside derivative kernels.
- **Analytical and exact derivative paths**: simple material scaling, laminate constitutive derivatives, mass coefficients, and some response derivatives are handled explicitly.
- **Adjoint solvers**: static response sensitivities in SOL 101 and buckling eigenvalue sensitivities in SOL 105 are assembled from global adjoint equations.
- **Optimization drivers**: thickness and grouped sizing optimization are implemented for compliance minimization, buckling maximization, and selected mass minimization routes.
- **SOL 200-lite translation**: a supported subset of Nastran-style optimization cards is translated into the internal grouped sizing optimizer while preserving design-variable grouping semantics.

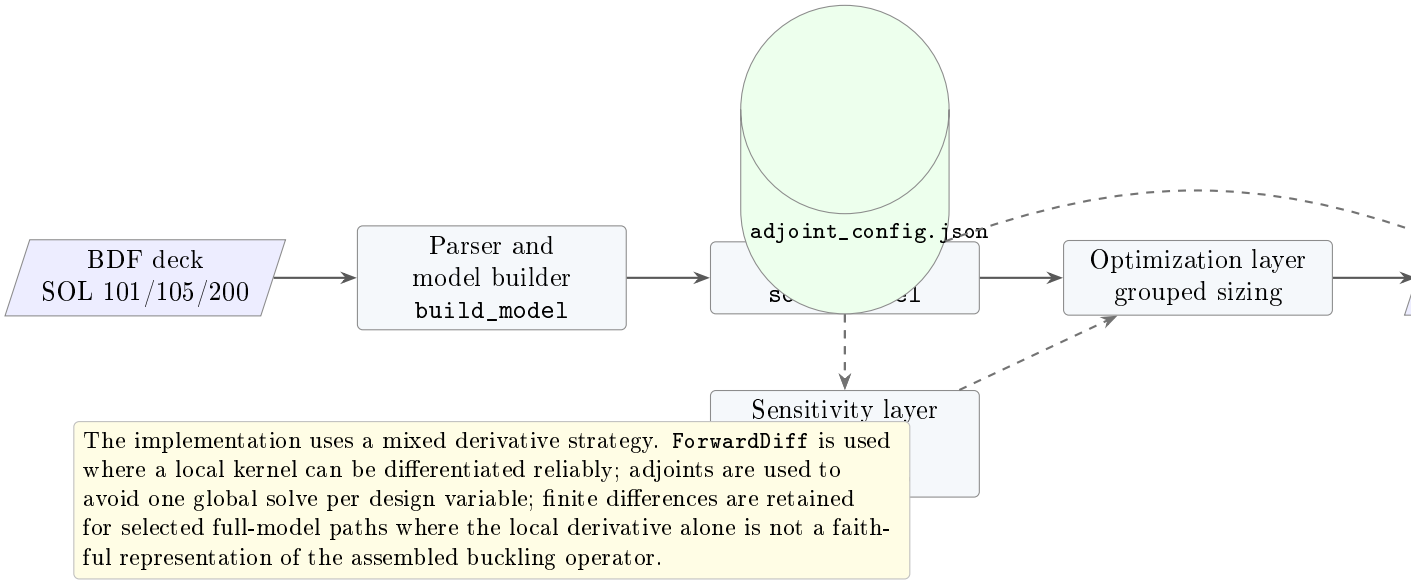


Figure 1: Implemented sensitivity and optimization stack.

2 Source Modules and Responsibilities

3 Automatic Differentiation Strategy

3.1 What "AD" Means in JFEM

The implemented automatic differentiation strategy is targeted. JFEM does not currently run whole-program AD through the entire solver. Instead, `ForwardDiff` is used inside local derivative kernels, especially for element stiffness contributions where the input dimension is small and the output can be contracted into the required global sensitivity quantity.

This choice is deliberate:

Source file	Primary responsibility
`src/solver/dKdx.jl`	Computes grouped pseudo-load vectors $\partial K/\partial x\, u$. Contains the design-variable registry and dispatches to analytical, AD, laminate-exact, element-FD, or full-model-FD derivative paths.
`src/solver/adjoint.jl`	Implements SOL 101 static adjoint sensitivities for displacement, shell stress, shell force, shell moment, and KS-aggregated von Mises responses.
`src/solver/adjoint_buckling.jl`	Implements SOL 105 buckling eigenvalue adjoints, including geometric stiffness derivatives, full-model fallback paths, and diagnostics.
`src/solver/optimize_thickness.jl`	Implements optimization drivers for thickness and sizing variables, including grouped sizing used by the SOL 200-lite translator.
`src/JFEMSolver.jl`	Dispatches by solution type, exposes package-level wrappers, translates the supported SOL 200 subset, and routes optimization to the grouped optimizer.
`src/main.jl`	Convenience CLI/API entry point. Parses BDF, solves, exports, then optionally runs the adjoint sidecar workflow if `adjoint_config.json` is present.
`src/Export.jl` and `src/MarkdownReport.jl`	Export optimization payloads, card inventories, JSON data, binary data, and human-readable reports.

Table 1: Main implementation files for AD, adjoint, optimization, and SOL 200.

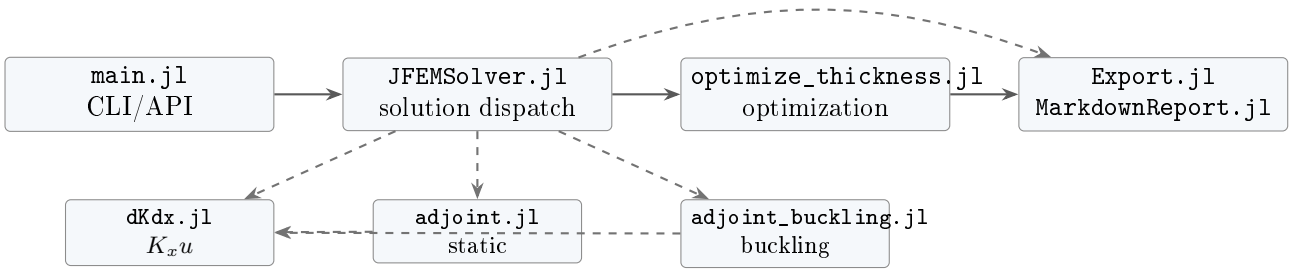


Figure 2: Module-level dependency map.

- Whole-model AD through sparse assembly, boundary condition elimination, factorization, and eigenvalue solves would be fragile and expensive.
- Local element AD gives accurate derivatives for nonlinear or coupled element formulas without requiring hand-derived expressions for every scalar path.
- Global adjoints then use those local derivatives efficiently, requiring one adjoint solve per scalar response instead of one forward solve per design variable.

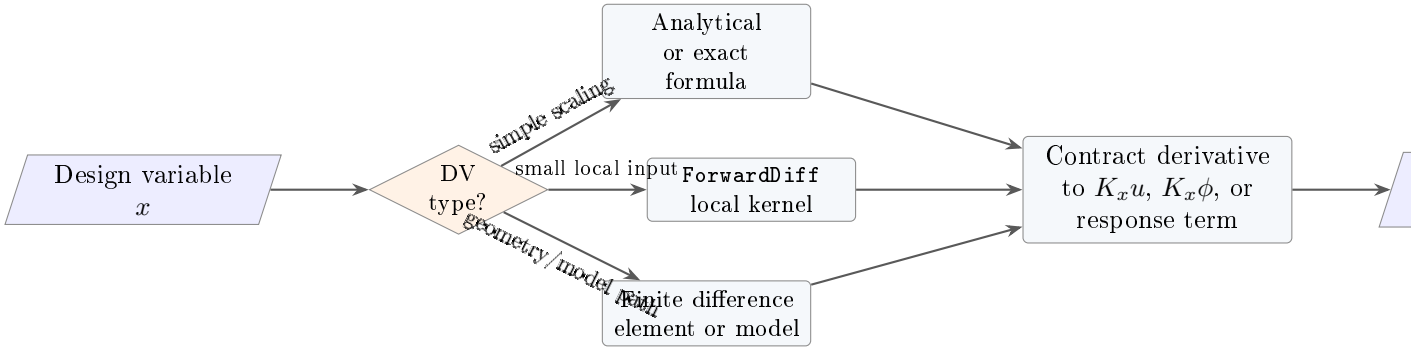


Figure 3: Derivative dispatch philosophy.

3.2 Design Variable Registry

The design-variable registry in `src/solver/dKdx.jl` maps each supported design-variable family to a default backend and group labeling convention.

Table 2: Implemented derivative families in `dKdx.jl`.

Design variable	Default method	Group label	Notes
`shell_thickness`	`ad_forward`	`PID_n`	Differentiates shell element stiffness with respect to property thickness. Falls back to element finite difference when needed.
`material_E`	`analytical`	`MID_n`	Uses stiffness proportionality with Young's modulus where applicable.
`material_NU`	`ad_forward`	`MID_n`	Differentiates stiffness with respect to Poisson ratio.
`bar_area`	`ad_forward`	`PID_n`	Supports bar/beam area sizing through property groups.
`pcomp_ply_thickness`	`laminate_exact`	`PID_n`	Uses laminate constitutive derivatives when available, with CLT finite difference fallback.
`pcomp_ply_angle`	`laminate_exact`	`PID_n`	Laminate angle derivative path for ply-level design.

Design variable	Default method	Group label	Notes
`node_coord`	`full_model_fd`	`GRID_g_c`	Geometry sensitivity through full-model perturbation.
`topology_density`	`analytical`	`EID_n`	Density-style derivative path for SIMP-like topology experiments.

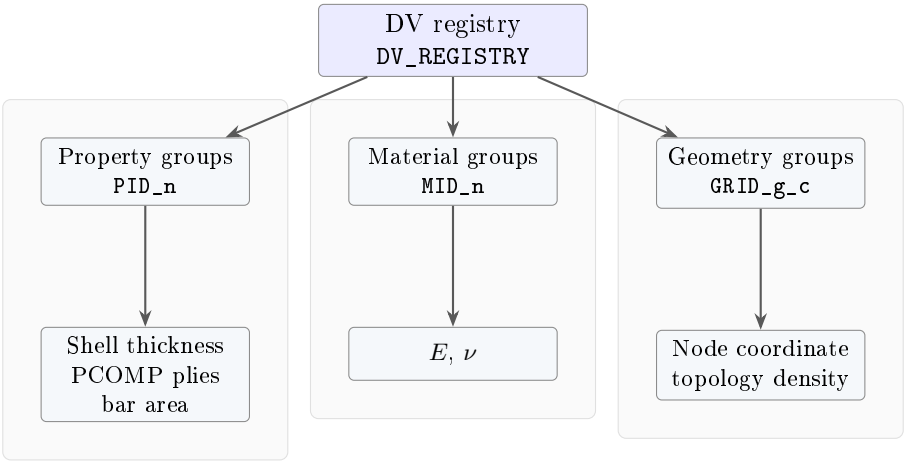


Figure 4: Registry-to-group labeling model. The group labels are the keys used in adjoint JSON output and grouped optimization.

3.3 Derivative Backend Selection

Most derivative functions accept a design-variable dictionary. The optional `method` field can override the default backend. This is useful for verification and for cases where exact local derivatives are not a faithful representation of the global assembled operator.

4 Static Adjoint Sensitivity for SOL 101

4.1 Mathematical Form

For a linear static problem

$$K(x)u(x) = F,$$

and a scalar response $r(u, x)$, the implemented adjoint equation is

$$K\lambda = \frac{\partial r}{\partial u}.$$

For the currently implemented design variables, the applied load is assumed not to depend on x, so the total sensitivity is

$$\frac{dr}{dx} = \left.\frac{\partial r}{\partial x}\right|_u - \lambda^T \frac{\partial K}{\partial x} u.$$

The explicit term is zero for pure displacement responses and nonzero for stress, shell force, shell moment, and aggregated stress responses.

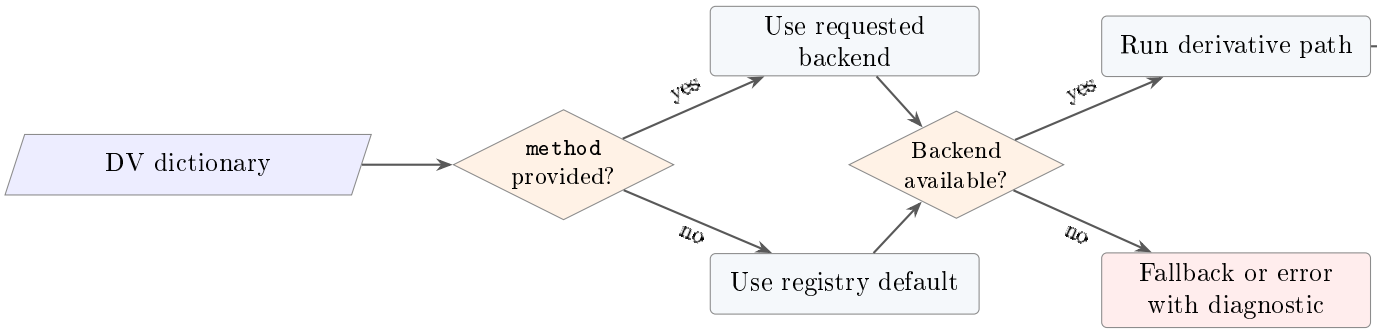


Figure 5: Backend override and fallback pattern.

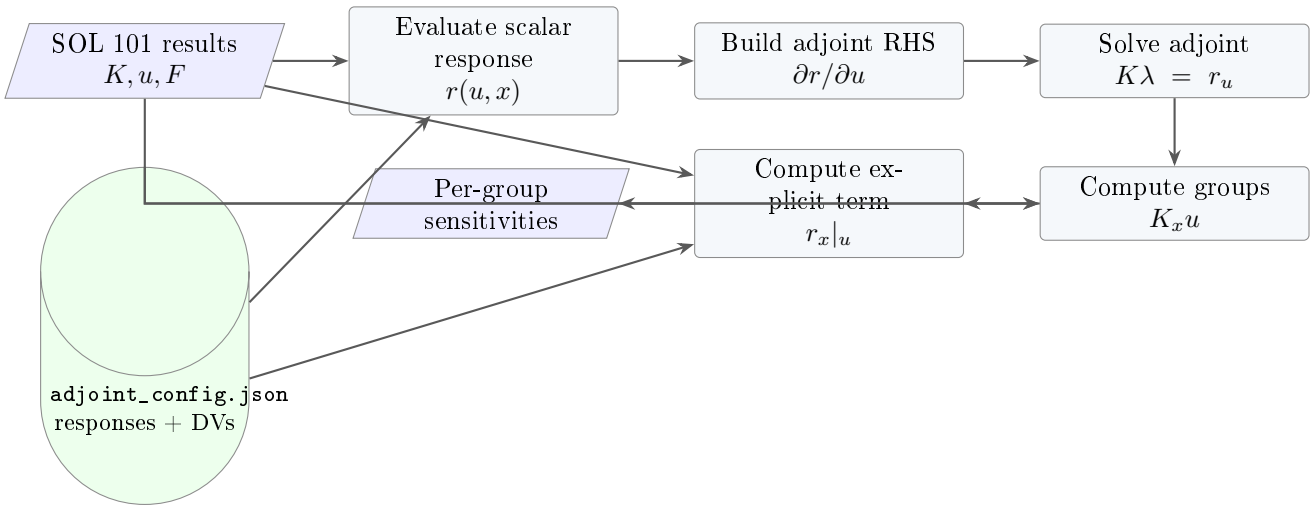


Figure 6: SOL 101 static adjoint workflow.

Response type	**Implemented behavior**
`displacement`	Returns a grid/component displacement and uses a unit vector as the adjoint RHS.
`von_mises`	Evaluates top or bottom shell centroid plane-stress von Mises response.
`shell_force_nx`, `ny`, `nxy`	Evaluates membrane force resultants from shell centroid strains.
`shell_moment_mx`, `my`	Evaluates bending moment resultants from shell curvature.
`ks_von_mises`	Computes a differentiable KS aggregate over selected shell elements or all shell elements. Uses a softmax-like weight vector for the adjoint RHS.

Table 3: Static adjoint response types implemented in the static adjoint source file.

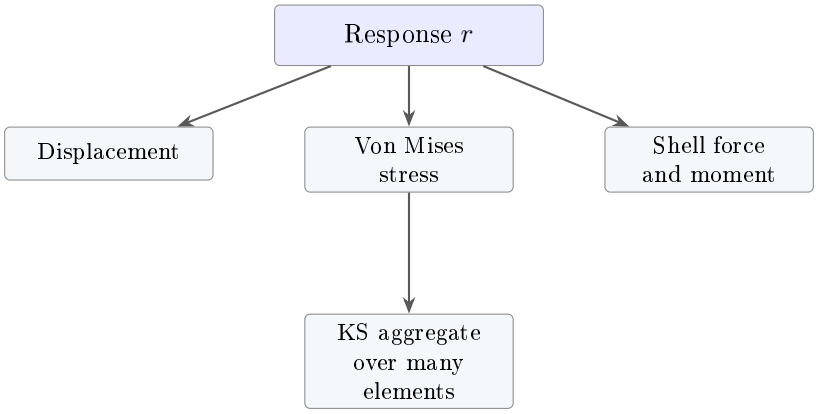


Figure 7: Implemented static response families.

4.2 Supported Static Response Types

4.3 KS-Aggregated Stress

For a vector of normalized element stress responses $g_i = \sigma_i/\sigma_0$, the KS response is implemented in a numerically stable shifted form:

$$KS(g) = g_{\max} + \frac{1}{\rho} \log \left(\sum_i \exp(\rho(g_i - g_{\max})) \right).$$

The corresponding weights are

$$w_i = \frac{\exp(\rho(g_i - g_{\max}))}{\sum_j \exp(\rho(g_j - g_{\max}))}.$$

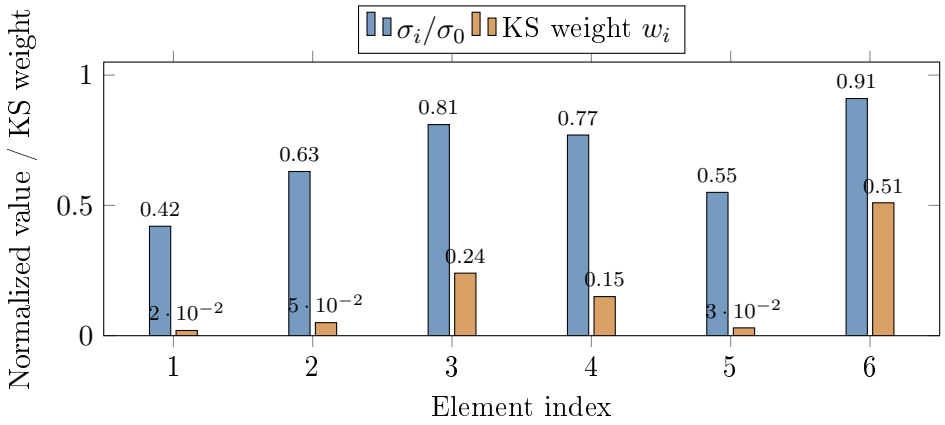


Figure 8: Conceptual view of KS aggregation. The largest stresses dominate the gradient while all elements remain differentiably active.

5 Buckling Adjoint Sensitivity for SOL 105

5.1 Implemented Sensitivity Equation

The buckling adjoint implementation works with the JFEM sign convention used by the SOL 105 eigenproblem. For an eigenvalue λ, mode shape ϕ, static displacement u, global stiffness K, and

geometric stiffness $K_g(u)$, the code evaluates grouped sensitivities in the form

$$\frac{d\lambda}{dx} = -\frac{\phi^T K_x \phi + \lambda\, \phi^T K_{g,x}|_u \phi - \psi^T K_x u}{\phi^T K_g \phi},$$

where the buckling adjoint vector solves

$$K\psi = \lambda \frac{\partial(\phi^T K_g(u)\phi)}{\partial u}.$$

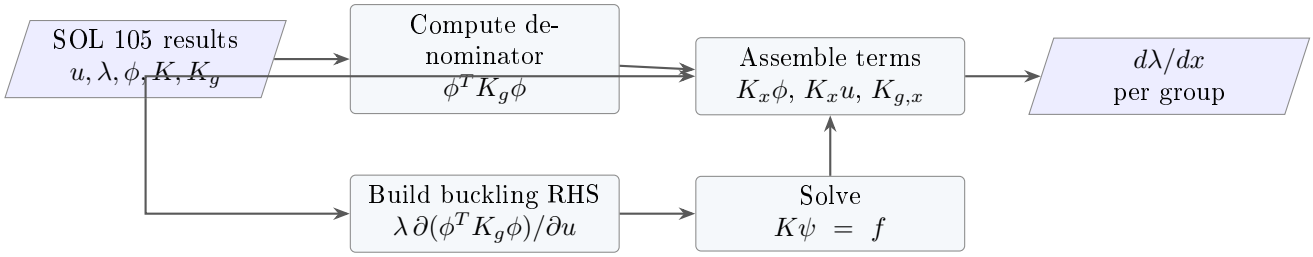


Figure 9: SOL 105 buckling adjoint workflow.

5.2 Why Buckling Sensitivity Needs More Paths

Buckling sensitivity is more demanding than static sensitivity because both the structural stiffness and geometric stiffness matter. The geometric stiffness depends on the stress state induced by the static solution, and the static solution itself changes with the design variable. The implementation therefore uses local derivative paths where reliable, but also includes full-model finite difference paths for groups that need assembled-operator consistency.

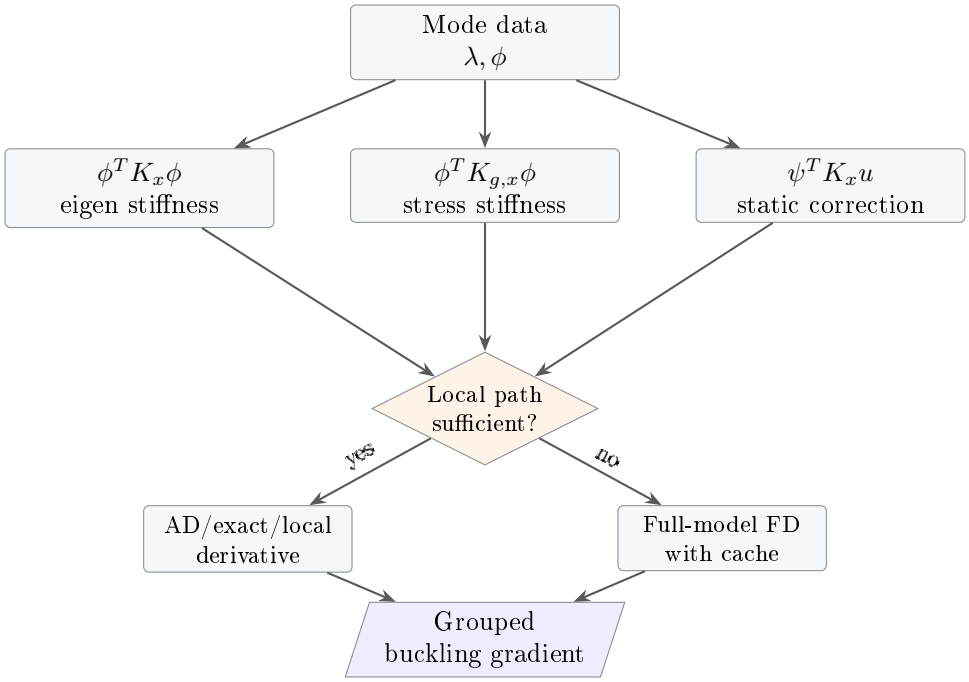


Figure 10: Buckling sensitivity combines multiple derivative terms and can select full-model fallback paths for difficult groups.

5.3 Buckling Diagnostics

The buckling adjoint result includes `path_diagnostics` and `path_summary`. These fields record whether a design variable used local AD/exact paths, full-model $\partial K/\partial x$, full-model $\partial K_g/\partial x$, or full sensitivity finite difference. This is important for traceability because buckling gradients can mix several derivative mechanisms in a single run.

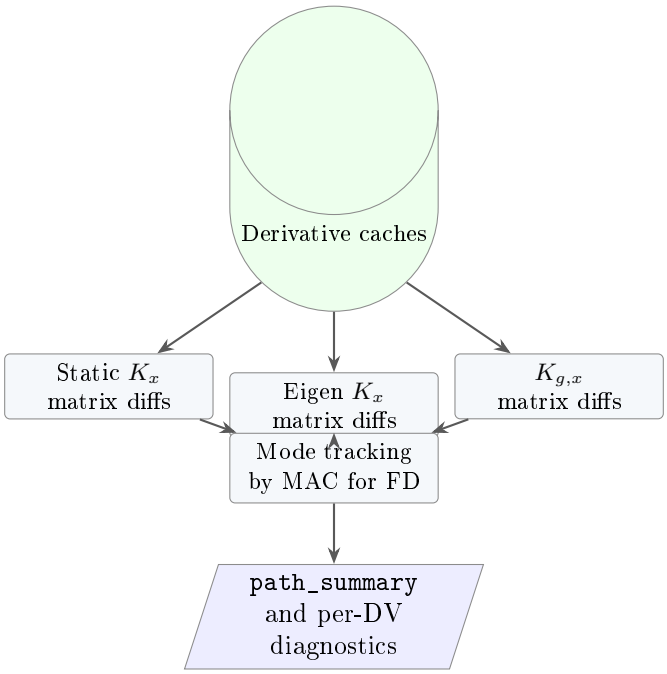


Figure 11: Buckling derivative caches and diagnostics.

6 Adjoint Configuration Sidecar

6.1 Execution Trigger

The convenience entry point in `src/main.jl` automatically looks for a file named `adjoint_config.json` in the same folder as the input deck. If the file exists and the solved case is SOL 101, `solve_adjoint` is called. If the solved case is SOL 105, `solve_adjoint_buckling` is called. The resulting JSON file is written as

`<case-name>.ADJOINT.JSON`.

6.2 Example Static Adjoint Configuration

Listing 1: Example `adjoint_config.json` for SOL 101.

```
{
  "responses": [
    {
      "id": "tip_z",
      "type": "displacement",
      "subcase": 1,
      "grid": 1001,
      "dof": 3
    },
    {
      "id": "skin_vm_ks",
```

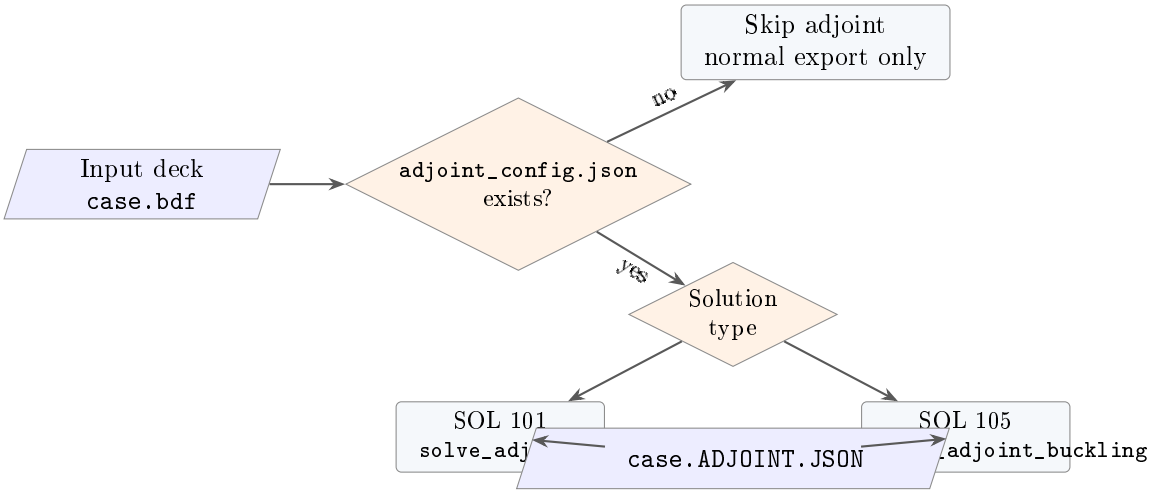


Figure 12: Automatic adjoint sidecar workflow.

```
      "type": "ks_von_mises",
      "subcase": 1,
      "eids": "all",
      "surface": "top",
      "sigma_ref": 1.0,
      "rho": 50.0
    }
  ],
  "design_variables": [
    {
      "id": "skin_t",
      "type": "shell_thickness",
      "pids": [10, 11, 12]
    },
    {
      "id": "aluminum_E",
      "type": "material_E",
      "mids": [1]
    }
  ]
}
```

6.3 Example Buckling Adjoint Configuration

Listing 2: Example `adjoint_config.json` for SOL 105.

```
{
  "responses": [],
  "design_variables": [
    {
      "id": "panel_t",
      "type": "shell_thickness",
      "pids": [101, 102]
    },
    {
      "id": "spar_area",
      "type": "bar_area",
      "pids": [201]
    }
```

```
  ]
}
```

For SOL 105, the implemented buckling adjoint computes sensitivities for each returned buckling mode, so scalar response entries are not needed in the same way as static adjoint responses.

7 Optimization Drivers

7.1 Implemented Optimization Families

The primary optimization implementation is in `src/solver/optimize_thickness.jl`. It contains both older direct thickness optimization and newer grouped sizing routes that preserve SOL 200 design variable grouping.

Function	Role
`optimize_thickness`	Element thickness optimization. Supports `:min_compliance` and `:max_buckling` objectives with a mass/volume fraction constraint.
`optimize_sizing`	General sizing route for supported sizing variables such as shell thickness and bar area.
`optimize_grouped_sizing`	Grouped design-variable route used by SOL 200-lite. Preserves one DESVAR controlling multiple properties/elements.
`optimize_grouped_static_response`	Minimize a selected static response under a mass target.
`optimize_grouped_static_constraints`	Handle multiple upper-bound static response constraints.
`optimize_grouped_single_variable_mass_minimization`	Single-variable mass minimization-style mass minimization with static constraints.
`optimize_grouped_static_response_mass_minimization`	Main grouped mass minimization route with outer bisection on mass target and inner grouped response optimization.

Table 4: Optimization functions implemented in the sizing optimization source file.

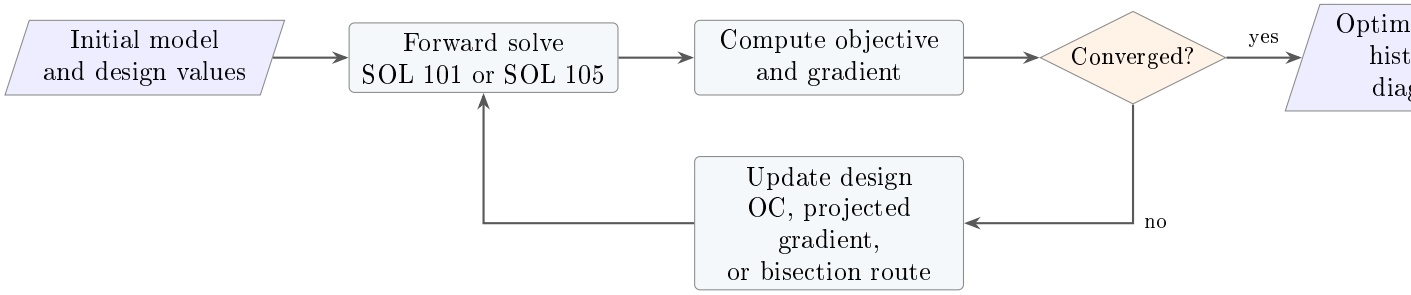


Figure 13: Generic optimization loop.

7.2 Objective and Constraint Concepts

The implemented optimization routes are practical engineering optimizers rather than a full general nonlinear programming framework. The key supported objective families are:

- **Minimize compliance**: static stiffness improvement in SOL 101.
- **Maximize buckling eigenvalue**: increase first or governing buckling load factor in SOL 105.
- **Minimize mass**: reduce mass subject to selected static response upper bounds on the supported SOL 200-lite subset.

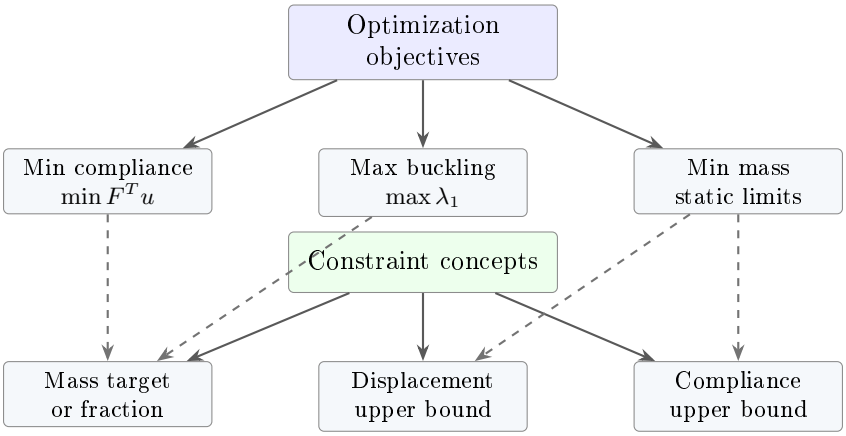


Figure 14: Optimization objectives and constraints currently represented in the implemented routes.

7.3 Grouped Sizing

The grouped sizing route is central to the SOL 200-lite implementation. A single design variable can control multiple properties; each property maps to many elements; each element may be split internally for analysis but remains associated with the original design group.

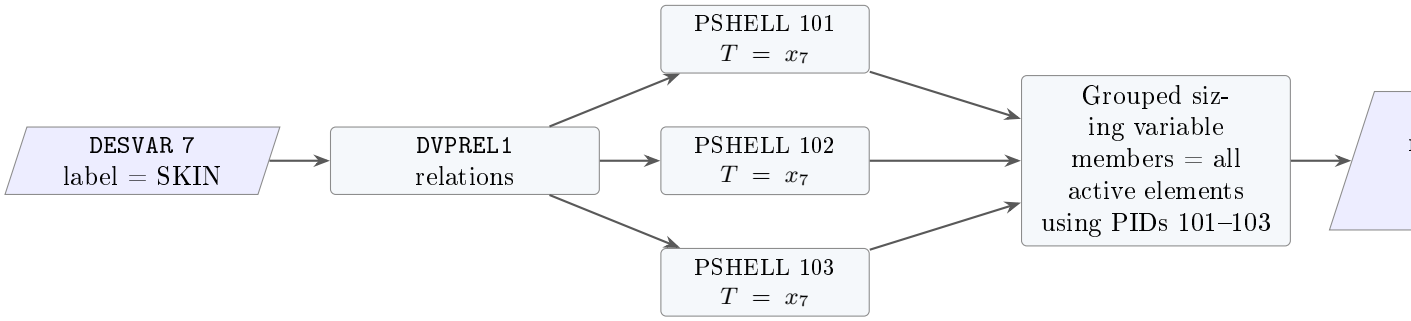


Figure 15: Grouped sizing preserves DESVAR semantics while allowing element-level analysis and sensitivity accumulation.

8 SOL 200-Lite Implementation

8.1 Purpose

The SOL 200-lite implementation is a translation layer. It does not attempt to reproduce every Nastran SOL 200 feature. Instead, it recognizes a useful subset of optimization cards and translates them into the internal grouped sizing optimizer.

The route is invoked automatically when the model solution type canonicalizes to SOL 200. The result dictionary then has `sol_type = 200`, `analysis_type = "SOL200_LITE_OPTIMIZATION"`, an `optimization` payload, a `route_summary`, and the final forward analysis results.

8.2 Supported Card Contract

The implemented subset is strict by design. Unsupported card combinations fail with explicit errors rather than silently changing the optimization problem.

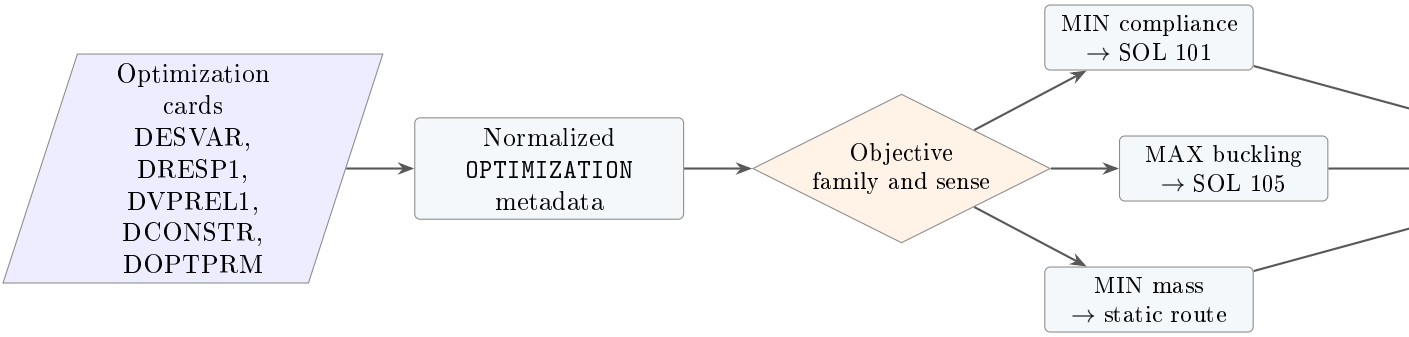
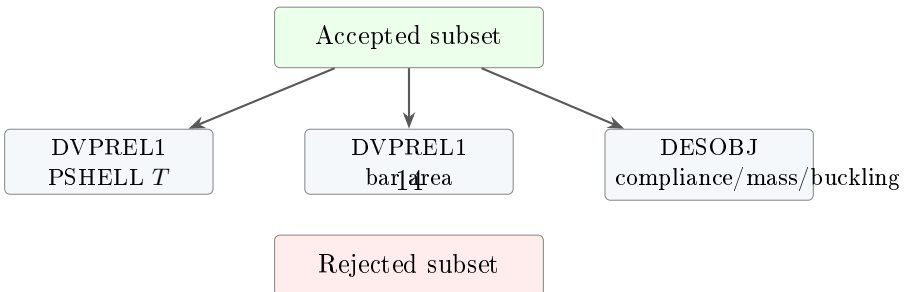


Figure 16: SOL 200-lite translation route.

Table 5: SOL 200-lite supported subset.

Feature	Supported	Current restrictions
`DESOBJ`	`MIN` compliance, `MIN` weight/mass on a narrow static subset, or `MAX` buckling eigenvalue.	Other objective families and senses are rejected.
`DRESP1`	Compliance, displacement, mass/weight, buckling eigenvalue candidate families.	Only the routed combinations above are accepted. Static mass minimization uses upper-bound compliance/displacement constraints.
`DVPREL1`	Shell thickness and bar area sizing relations.	One DESVAR per supported relation, unit coefficient, zero offset, and at most one active relation per property and sizing family.
`DESVAR`	Initial value, lower/upper bounds, and move limit.	Initial values must satisfy translated bounds. Grouped DESVAR initialization must be consistent across its relations.
`DCONSTR`	Mass upper bound for compliance/buckling objectives; static upper bounds for mass minimization.	Compliance/buckling routes currently require exactly one upper-bound mass constraint.
`DOPTPRM`	`DESMAX`, `CONV1`, and `DELX`.	`DELX`, when provided, must be positive.
`DESSUB/STATSUB`	Subcase selection for static and buckling routes.	Static route needs exactly one subcase or a global DESSUB selector. Buckling can retain the design subcase and its STATSUB static subcase.
`DVMREL1`	Parsed into metadata.	Material optimization through DVMREL is not yet supported in the SOL 200-lite route.



8.3 Route Selection

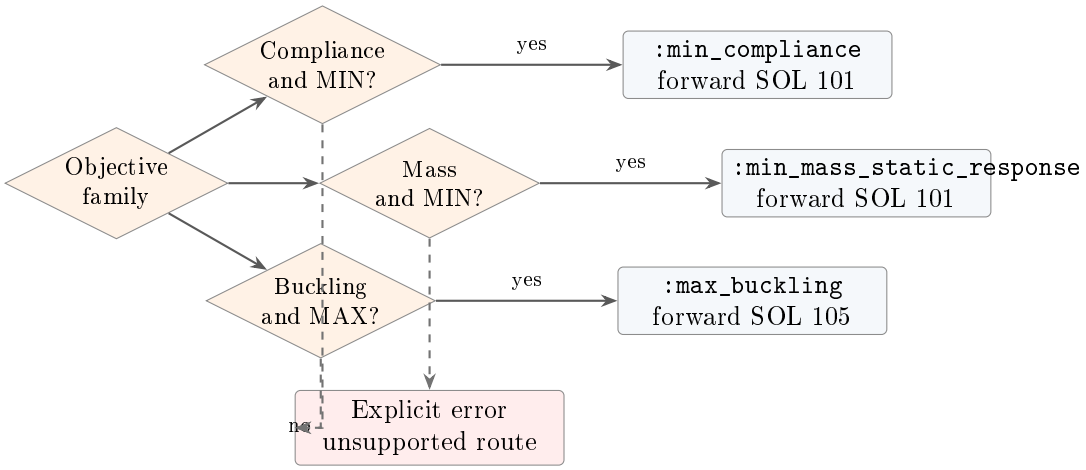


Figure 18: Objective route selection in `_sol200_lite_translate`.

9 Exported Data and Reports

9.1 Pipeline Outputs

The normal JFEM pipeline exports solver results according to the selected output flags. Optimization results are included in the result dictionary and can be exported in JSON and markdown reports. Compact Nastran-like HDF5 export is intentionally skipped for SOL 200 optimization at the current implementation state.

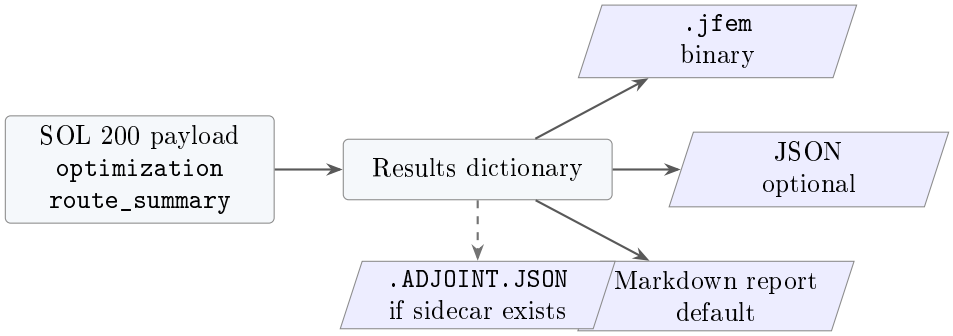


Figure 19: Relevant outputs for sensitivity and optimization workflows.

9.2 SOL 200 Result Dictionary Shape

The SOL 200-lite result has the following top-level concepts:

Listing 3: Conceptual SOL 200-lite result payload.

```
{
  "sol_type": 200,
  "analysis_type": "SOL200_LITE_OPTIMIZATION",
  "optimization": {
    "objective": "...",
    "iterations": [],
    "design_variables": {},
    "final_mass": 0.0,
```

```
    "model": {}
  },
  "route_summary": {
    "translated_objective": "...",
    "forward_sol_type": 101,
    "grouped_design_variable_count": 0
  },
  "forward_results": {},
  "solver_diagnostics": {}
}
```

10 Performance Considerations

10.1 Why Adjoint Methods Matter

If there are n_x design variables and one scalar response, a direct finite difference strategy needs approximately $2n_x$ forward solves for central differences. The adjoint strategy needs one forward solve plus one adjoint linear solve per response, with local or grouped derivative contractions for the design variables.

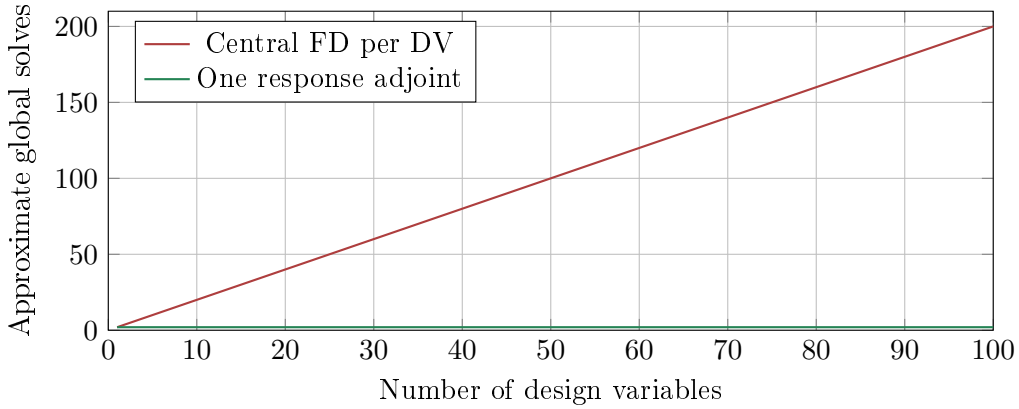


Figure 20: Adjoints decouple the number of global linear solves from the number of design variables for each scalar response.

10.2 Where AD Is Efficient

Local forward-mode AD is efficient when the number of independent variables is small. In JFEM, this is typically the case for one element property, one material scalar, or one ply scalar. It is not generally efficient for a full global vector of all design variables.

11 Implementation Maturity

12 Developer Notes and Extension Points

12.1 Adding a New Design Variable

The clean path for adding a design variable is:

1. Add the family to the derivative registry in `src/solver/dKdx.jl`.

2. Define group labels that match how the optimizer and JSON output should report sensitivities.

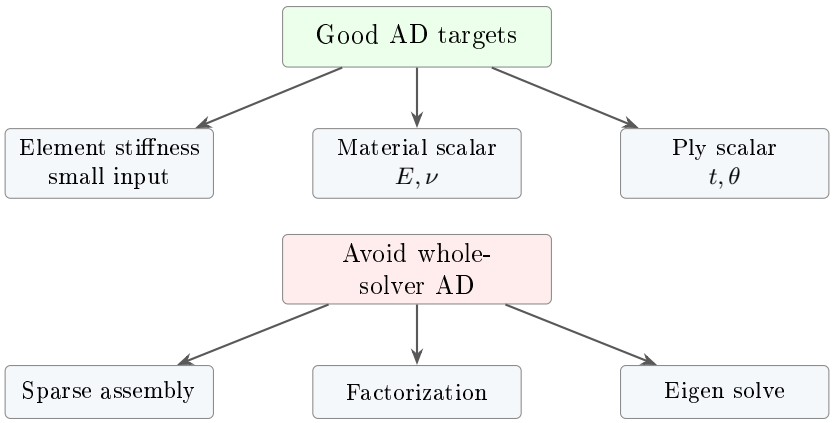


Figure 21: Targeted local AD is the intended implementation pattern.

Capability	**Status**	**Remarks**
Static adjoint for displacement	Implemented	Most direct response path; no explicit design derivative.
Static adjoint for shell stress/resultants	Implemented	Centroid top/bottom von Mises, force, and moment resultants are supported.
KS aggregated shell stress	Implemented	Useful for optimization constraints over many elements.
Buckling eigenvalue adjoint	Implemented	Includes full-model fallback diagnostics for difficult derivative groups.
Thickness and sizing optimization	Implemented	Compliance minimization and buckling maximization are available.
SOL 200-lite translator	Implemented subset	Strict supported subset focused on shell/bar sizing, mass constraints, and selected objective families.
Material optimization via DVMREL	Not yet routed	Cards can be present in metadata, but the SOL 200-lite route rejects DVMREL optimization.
Whole-program AD	Not implemented	Current design intentionally uses local AD plus adjoints.
General nonlinear programming	Not implemented	Current optimizers are specialized engineering routes, not a full NLP engine.

Table 6: Maturity summary for sensitivity and optimization features.

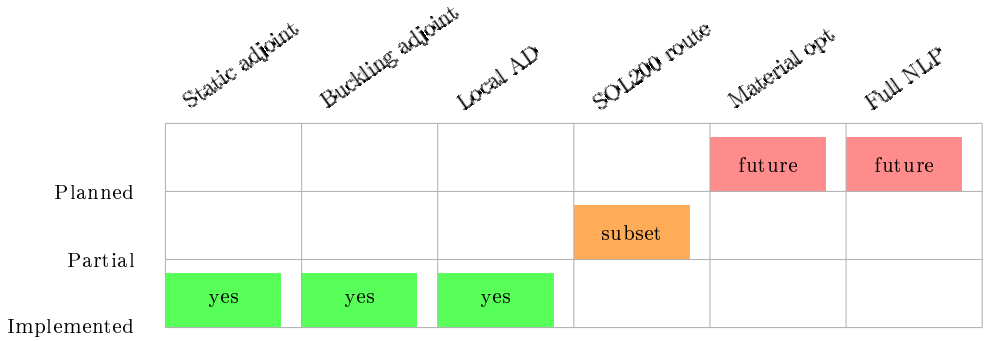


Figure 22: Feature maturity heatmap.

3. Implement $K_x u$ and, if needed, $K_x \phi$ support.
4. Add explicit response derivatives in `src/solver/adjoint.jl` if the response depends directly on the design variable at fixed displacement.
5. For SOL 105, decide whether $K_{g,x}$ can be local/exact or requires full-model fallback.
6. Add parser and SOL 200-lite translation support only after the core sensitivity path is verified.

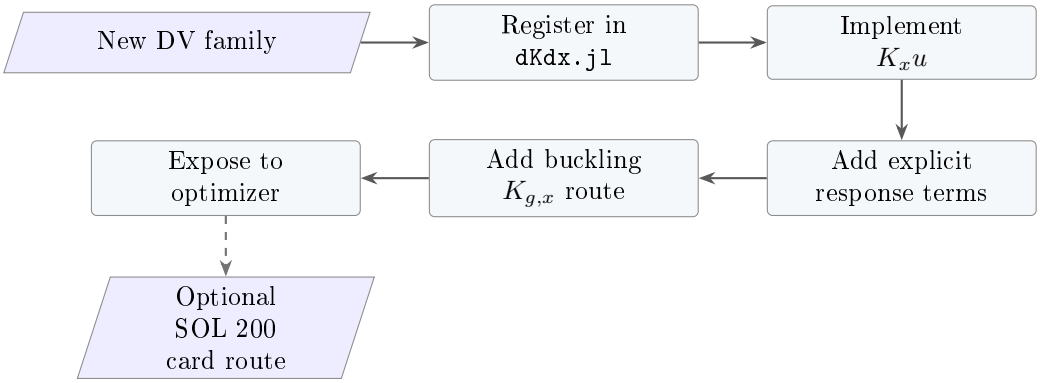


Figure 23: Recommended implementation order for a new design variable.

12.2 Adding a New Response

For a new static scalar response, implement:

- response evaluation $r(u, x)$;
- adjoint RHS $\partial r / \partial u$;
- explicit derivative $\partial r / \partial x|_u$, if nonzero;
- optional SOL 200-lite DRESP1 family translation.

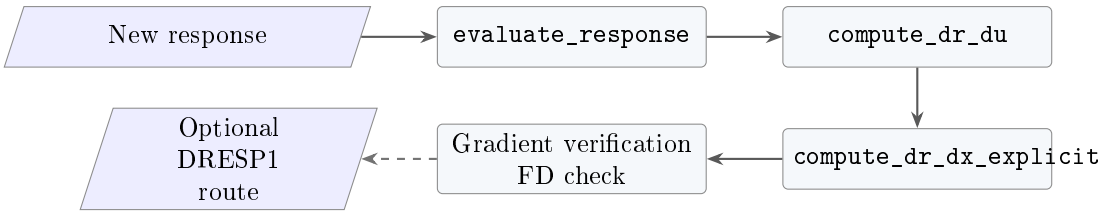


Figure 24: Recommended implementation order for a new static response.

12.3 Verification Philosophy

The sensitivity stack should be verified at several levels:

- **Element derivative check**: compare AD/exact $K_x u$ to element finite difference.
- **Global derivative check**: compare grouped adjoint sensitivity to full-model finite difference.
- **Optimization smoke test**: verify that objective and constraint histories move in the expected direction.
- **SOL 200 translation test**: verify that DESVAR grouping, bounds, DVPREL membership, and DCONSTR meaning are preserved.

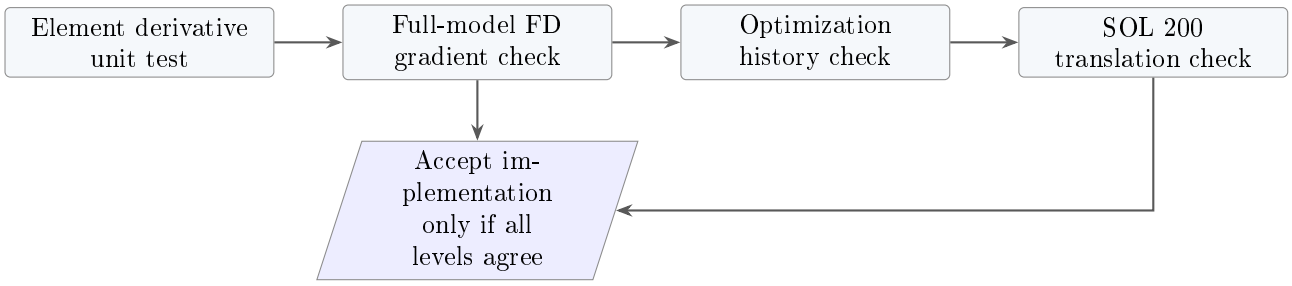


Figure 25: Recommended verification ladder.

13 Known Boundaries

The current implementation is strong enough for selected shell/bar sizing optimization workflows, but it should not be presented as complete Nastran SOL 200 parity. The most important boundaries are:

- Material design through `DVMREL1` is not routed by SOL 200-lite.
- General DVPREL expressions with multiple DESVARs, non-unit coefficients, or nonzero offsets are rejected.
- Rods, solids, concentrated masses, and arbitrary mass accounting are not supported in the strict SOL 200-lite mass scope.
- General NLP behavior, multiple objective functions, arbitrary nonlinear constraints, and optimizer algorithms such as full MMA/SQP are not yet implemented as production routes.
- Whole-program AD is not implemented; the AD capability is a local derivative mechanism feeding adjoints and optimization.

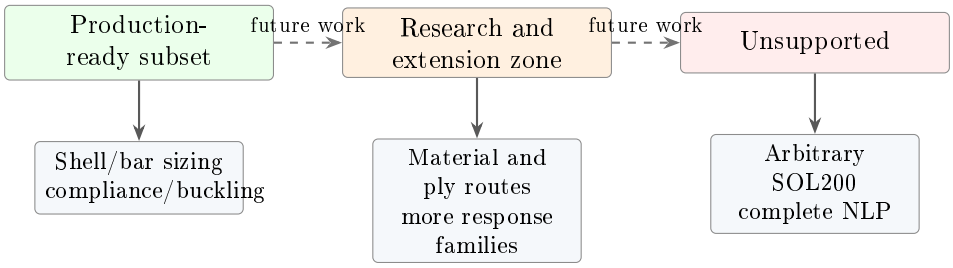


Figure 26: Current support boundary.

14 Recommended Future Work

1. **Formalize derivative verification tests**. Add automated element, global, and optimization gradient checks for every supported design variable and response family.
2. **Expand SOL 200-lite material routes**. Connect DVMREL1 to existing `material_E` and `material_NU` derivative families once mass and grouping semantics are defined.
3. **Consolidate full-model buckling fallback criteria**. Keep the fallback diagnostics but document and test every route selection rule.
4. **Introduce a production optimizer interface**. If needed, connect a more general optimizer package while preserving the current grouped sizing abstractions.

5. **Broaden response families**. Add additional DRESP1 candidates only after response evaluation, adjoint RHS, explicit derivative, and finite-difference verification exist.

6. **Keep SOL 101/SOL 105 parity guards active**. Optimization is only as reliable as the underlying forward solver and gradient parity.

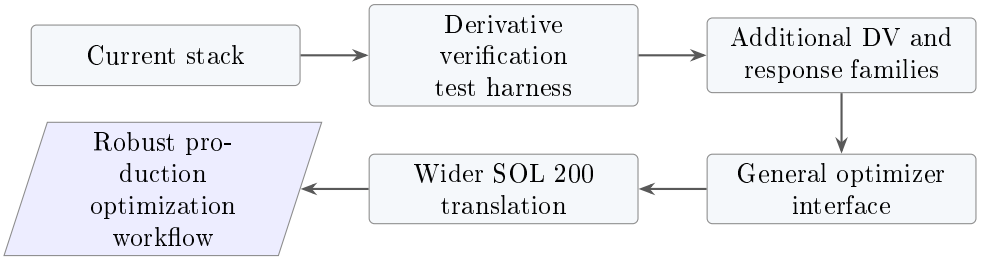


Figure 27: Suggested development roadmap.

15 Quick Reference

15.1 Static Adjoint Formula

$$K\lambda = r_u, \qquad r_x = r_x|_u - \lambda^T K_x u.$$

15.2 Buckling Adjoint Formula

$$K\psi = \lambda \frac{\partial(\phi^T K_g(u)\phi)}{\partial u}, \qquad \lambda_x = -\frac{\phi^T K_x \phi + \lambda \phi^T K_{g,x}|_u \phi - \psi^T K_x u}{\phi^T K_g \phi}.$$

15.3 Most Important Files

- `src/solver/dKdx.jl`: derivative registry and $K_x u$ kernels.
- `src/solver/adjoint.jl`: SOL 101 response adjoints.
- `src/solver/adjoint_buckling.jl`: SOL 105 eigenvalue adjoints.
- `src/solver/optimize_thickness.jl`: optimization drivers.
- `src/JFEMSolver.jl`: SOL 200-lite routing and public wrappers.
- `src/main.jl`: CLI/API pipeline and optional adjoint sidecar.

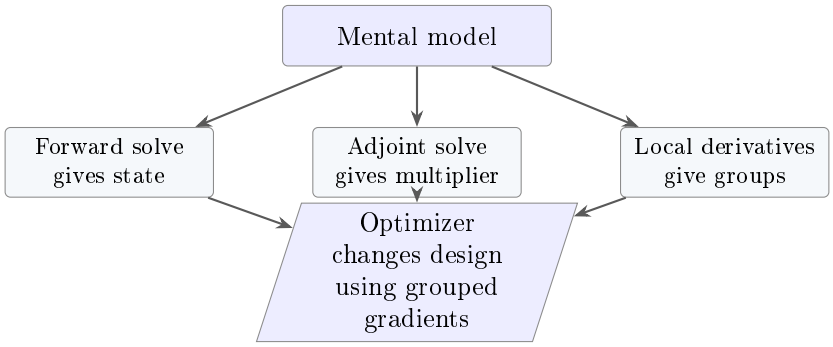


Figure 28: One-page mental model for the implemented features.